

OBJECTIVE-PRESERVING, OPTIMIZER-AWARE TASK SAMPLING FOR MULTI-TASK LLM POST-TRAINING

Anonymous authors

Paper under double-blind review

ABSTRACT

Multi-task LLM post-training uses task weights to specify capability priorities and task sampling to allocate compute. Conflating the two makes efficiency claims hard to interpret; global token averaging can also give long-response tasks unintended weight.

We present Gradient-Preserving Adaptive Sampling (GPAS) for multi-teacher on-policy distillation and multi-task reinforcement learning. Each update uses a task-homogeneous batch and gives every response equal weight. GPAS measures task gradients in a metric induced by the optimizer: it recovers raw-gradient sampling for SGD, while our LLM instantiation uses AdamW’s stored second moment. GPAS samples high-impact tasks more often and rescales every sampled gradient by its target weight divided by its sampling probability. The conditional expected gradient remains equal to the user-specified task mixture. The online sampler stores one score per task, and Cost-GPAS also uses measured task time.

In a controlled three-task experiment, GPAS attains a calculated error ratio of 0.698 and a Monte Carlo ratio of 0.697 relative to uniform sampling; the cost-aware proposal lowers its local compute proxy by 10.7%. The completed experiment verifies the estimator calculation. Four-teacher MOPD and three-task GRPO experiments will test the AdamW score, online tracking, teacher compatibility, and progress per token and GPU hour. The large-model experiments remain pending in this draft.

1 INTRODUCTION

Post-training one model on mathematics, code, instruction following, and science requires two decisions. Target weights state which capabilities the objective values. Sampling probabilities decide which task receives the next rollout, teacher call, and update. Keeping target weights and sampling probabilities separate makes both decisions visible.

Many systems use the sampling ratio as the task weight. Sampling a weak or cheap task more often then changes the model objective, so the resulting capability trade-off can come from the objective, the compute schedule, or both. A coupled design makes optimization efficiency difficult to measure.

Response length creates a second ambiguity. In a prompt-balanced mixed batch, global token averaging gives long-response tasks a larger coefficient. Recent MOPD work diagnoses this imbalance and changes domain loss weights (Gao et al., 2026). Our training objective gives each response equal weight inside a task-homogeneous batch.

Teacher selection is a prerequisite for OPD. A wide teacher–student gap can create extreme token log-ratios, whereas a narrow gap supplies little signal. Recent studies report stronger transfer at moderate distributional overlap (Ma et al., 2026a; Wang et al., 2026b; Ma et al., 2026b). We train teachers from the student initialization, audit them on student rollouts, and study task allocation only within the selected teacher set.

Suppose task i has target weight λ_i and sampling probability q_i . Multiplying its sampled gradient by λ_i/q_i keeps the conditional mean equal to the target mixture. The sampling probabilities can then allocate compute while the target mixture stays fixed.

Gradient-Preserving Adaptive Sampling (GPAS) applies classical importance sampling at the task-batch level. GPAS scores task gradients in a metric induced by the training optimizer, then assigns more probability to tasks with larger scores. The identity metric recovers raw-gradient sampling for SGD; for AdamW, we use the lagged preconditioner from its stored second moment. A moving average estimates each task score from completed updates. A probability floor limits stale scores and large importance weights. Cost-GPAS also uses measured task time.

The controlled experiment checks the sampling formulas with known gradient distributions and costs. The large-model experiments use frozen gradient banks to compare raw gradients, GPAS scores, and measured AdamW task steps. Paired training runs then measure teacher-loss progress per token and GPU hour. A GRPO experiment tests the same task sampler with reward-based updates.

The paper makes three contributions:

- We define a response-normalized task-batch interface that separates a fixed multi-task target from compute allocation in on-policy post-training.
- We derive task allocation in an optimizer-induced metric, recovering raw-gradient sampling for SGD and a lagged-preconditioned rule for AdamW. A cost-aware version also uses measured end-to-end task time, and both samplers estimate their scores online.
- We validate the estimator calculation and specify direct MOPD and GRPO tests that connect score quality to progress per token and GPU hour.

Section 3 defines the objective and sampling interface; Section 4 presents GPAS; Section 5 gives the MOPD and GRPO studies. The appendix contains proofs and implementation details.

2 RELATED WORK

Multi-task optimization. Multi-task methods often learn task weights or modify gradient directions to improve a capability trade-off (Sener & Koltun, 2019; Yu et al., 2020; Liu et al., 2024; Chen et al., 2018). GPAS starts from task weights supplied by the user. GPAS changes the frequency of task batches and keeps the weighted gradient fixed in expectation. A fixed target makes optimization efficiency measurable under one capability objective.

Multi-task RL for language models. Joint RL post-training can show uneven gradients and uneven task progress (Ramesh et al., 2026; Wu et al., 2025). Difficulty-based curricula sample weak or slowly improving tasks more often (Corrado et al., 2026). Multi-Task GRPO learns robustness-oriented task weights and preserves its realized batch ratios (Ramesh et al., 2026). Our experiments keep the target task weights fixed. A corrected gap sampler measures the value of a loss-gap signal under the same target objective. An uncorrected version measures the capability shift caused by changing task priorities.

Multi-teacher on-policy distillation. MOPD routes student rollouts to domain teachers and trains one shared student (Ma et al., 2026a). Related systems use task-specific teacher interfaces and language-specialized teachers (Ma et al., 2026b; Xie et al., 2026). Open-MOPD identifies response length, teacher-student gap, and reused rewards as sources of capability imbalance and changes domain loss weights (Gao et al., 2026). Other OPD studies report unreliable token guidance under large teacher-student distribution mismatch (Wang et al., 2026b;a). Same-origin teachers and moderate next-token overlap improve training stability while preserving a useful learning signal (Ma et al., 2026a;b). Our MOPD protocol uses four same-origin teachers, task-homogeneous fixed-token batches, and one optimizer use per rollout. GPAS changes the frequency of complete task batches and applies an inverse sampling factor. Cost-GPAS adds measured GPU time for generation, teacher evaluation, and model updates.

Importance sampling for stochastic optimization. Sampling examples from their gradient size and applying an inverse probability factor can reduce stochastic-gradient variance (Alain et al., 2015; Katharopoulos & Fleuret, 2018). Cost-aware SGD further samples in proportion to gradient scale divided by the square root of evaluation cost and has been applied to GRPO (Mohri et al., 2026). Non-

uniform sampling can interact nonlinearly with adaptive optimizers (Lahire, 2023). PiKE adapts data-mixture weights using gradient magnitude, variance, and interactions (Li et al., 2025).

Cost-Aware GRPO decides which already-generated sequences enter policy updates. GPAS decides which task receives the next rollout, teacher call, and optimizer step while preserving a user-specified multi-task objective. GPAS evaluates complete task-batch gradients in an optimizer-induced metric and estimates task scores from completed updates. Its SGD instance recovers classical gradient-norm sampling; its AdamW instance uses a lagged preconditioner. Cost-GPAS applies the published cost-aware sampling identity to the same task-batch setting. Our contribution lies in fixed-target multi-task allocation, response-normalized task batches, and optimizer-aware online scores.

3 TASK OBJECTIVE AND SAMPLING

Task allocation starts with a clear definition of one task gradient. We first average responses inside a task-homogeneous batch. We then sample one task while preserving the user-selected mixture of task gradients.

3.1 DEFINE A COMPARABLE TASK GRADIENT

Consider M tasks. Task i supplies prompts from $\mathcal{D}_i$ and has target weight λ_i , with $\sum_i \lambda_i = 1$. One training draw selects a task and builds a task-homogeneous batch. For response j , let L_j be its number of valid response tokens and let $g_{j,t}$ be its token gradient. The task-batch gradient is

$$G_i = \frac{1}{n} \sum_{j=1}^n \frac{1}{L_j} \sum_{t=1}^{L_j} g_{j,t}. \quad (1)$$

Each response receives equal weight inside its task. A long response supplies more token observations and keeps the same total coefficient as a short response. Every task batch targets a shared number of valid response tokens. Gradient variance can still differ by task, and GPAS measures task-dependent variance. GPU time is recorded separately because prompt length, generation, and teacher evaluation have task-dependent costs.

For MOPD, $g_{j,t}$ is the sampled-token policy gradient. For GRPO, $g_{j,t}$ comes from the clipped policy surrogate. Prefix sampling, token truncation, reward clipping, and gradient clipping are part of the implemented G_i . The sampler preserves the mean of the implemented G_i . Appendix B.1 gives the OPD gradient, and Appendix B.2 describes the GRPO gradient. When the trainer applies task-level clipping, G_i denotes the clipped gradient from Equation 1.

The mean gradient of task i and the target multi-task gradient are

$$h_i = \mathbb{E}[G_i], \quad h = \sum_i \lambda_i h_i. \quad (2)$$

For frozen teachers, λ states the desired distillation mixture.

A full estimate of h requires one complete batch from every task. Sampling one task uses one rollout and one teacher or reward call per optimizer step. The response-normalized G_i provides a common unit for task sampling.

3.2 SEPARATE TASK PRIORITIES FROM SAMPLING FREQUENCY

Let q_i be the probability that task i supplies the next batch. After sampling task I , the trainer sends

$$\hat{g} = \frac{\lambda_I}{q_I} G_I, \quad I \sim q, \quad (3)$$

to the optimizer. When $q_i = \lambda_i$, the multiplier equals one. Other sampling probabilities use the multiplier to keep the target gradient fixed.

Proposition 1 (Conditional objective preservation). *Let $\mathcal{H}_t$ contain the training history before step t , let q_t be determined by $\mathcal{H}_t$ with $q_{t,i} > 0$, and define $h_{t,i} = \mathbb{E}[G_{t,i} \mid \mathcal{H}_t, I_t = i]$. The corrected update preserves the target conditional mean.*

Proof. Because q_t is fixed conditional on $\mathcal{H}_t$, averaging over the task draw and its fresh batch gives

$$\mathbb{E}[\hat{g}_t \mid \mathcal{H}_t] = \sum_i q_{t,i} \frac{\lambda_i}{q_{t,i}} \mathbb{E}[G_{t,i} \mid \mathcal{H}_t, I_t = i] = \sum_i \lambda_i h_{t,i}. \quad (4)$$

□

The weights λ define capability priorities. The probabilities q allocate task batches. Equation 4 assigns a separate role to each quantity. The equality concerns the gradient passed to the optimizer. Stateful optimizers can still produce different parameter paths under different task orders.

An online sampler can use earlier gradients, measured costs, and optimizer states to choose the next q . Every task keeps positive probability, and each new task batch uses fresh rollout randomness, so Proposition 1 applies at every optimizer step.

Task allocation now has a measurable objective: choose q so that one corrected task batch estimates h accurately at low cost. Section 4 derives GPAS from gradient scale and Cost-GPAS from gradient scale plus GPU time. Experiment E1 tests the definition of G_i , E2 tests the corrected estimator, and the end-to-end MOPD experiment measures training efficiency.

4 GRADIENT-PRESERVING ADAPTIVE SAMPLING

Section 3.2 fixes the target task mixture and leaves the sampling probabilities available for compute allocation. Every expectation in this section conditions on the model, optimizer, and sampler state before the next task is selected.

4.1 SAMPLE TASKS IN THE OPTIMIZER METRIC

Let A_t be a linear map determined by the optimizer state before task selection. It maps gradients into the local update geometry whose estimation error we seek to reduce. Conditional on the training history, A_t is fixed. Suppressing the step index, define

$$\tilde{G}_i = A_t G_i, \quad s_i = \sqrt{\mathbb{E}\|\tilde{G}_i\|_2^2}. \quad (5)$$

For SGD, $A_t = I$; its scalar learning rate does not change the proposal. Thus the SGD instance is Raw GPAS. For AdamW, raw gradient norms can be misleading because parameter coordinates have different effective step sizes. Let $\bar{v}_{\text{prev}}$ be AdamW’s stored, bias-corrected second moment before task selection. Its lagged metric is

$$A_t^{\text{AdamW}} G_i = \frac{G_i}{\sqrt{\bar{v}_{\text{prev}}} + \epsilon_{\text{Adam}}}, \quad (6)$$

where division is element-wise. The LLM experiments use this instance and call it GPAS unless stated otherwise.

For the corrected task gradient in Equation 3, the optimizer-metric mean-squared error is

$$\mathbb{E}\|A_t(\hat{g} - h)\|_2^2 = \sum_i \frac{\lambda_i^2 s_i^2}{q_i} - \|A_t h\|_2^2. \quad (7)$$

The part that depends on the proposal is

$$V(q) = \sum_i \frac{\lambda_i^2 s_i^2}{q_i}. \quad (8)$$

Proposition 2 (Optimizer-metric-optimal task proposal). *If $\lambda_i s_i > 0$ for every task, $V(q)$ over the probability simplex is minimized by $q_i^* = \lambda_i s_i / \sum_j \lambda_j s_j$.*

Proof. Cauchy–Schwarz and $\sum_i q_i = 1$ give

$$\left(\sum_i \lambda_i s_i \right)^2 = \left(\sum_i \sqrt{q_i} \frac{\lambda_i s_i}{\sqrt{q_i}} \right)^2 \leq \left(\sum_i q_i \right) V(q) = V(q). \quad (9)$$

Equality requires $q_i \propto \lambda_i s_i$, which proves the claim. □

The identity metric makes the result the classical SGD gradient-norm rule. For AdamW, GPAS uses the stored state because the score must be available before task selection. This lagged score approximates the task-dependent part of the next AdamW step; E3 compares it with a measured AdamW task step.

4.2 INCLUDE MEASURED TASK COST

Fixed-token task batches can have different runtimes. Let c_i be the measured time for generation, teacher or reward evaluation, actor training, and the optimizer step. Cost-GPAS minimizes the local compute score $J(q) = (\sum_i q_i c_i) V(q)$.

Corollary 1 (Cost-aware task proposal). *If additionally $c_i > 0$, the minimizer of $J(q)$ is $q_i^* = (\lambda_i s_i / \sqrt{c_i}) / \sum_j (\lambda_j s_j / \sqrt{c_j})$.*

Proof. Cauchy–Schwarz gives $J(q) \geq (\sum_i \lambda_i s_i \sqrt{c_i})^2$, with equality exactly when $q_i \propto \lambda_i s_i / \sqrt{c_i}$. $\square$

This applies the cost-aware sampling rule of Mohri et al. (2026) to complete task batches in the selected optimizer metric. $J(q)$ is a local training signal. E2 measures it on held-out gradient banks, and the end-to-end experiment measures weighted teacher loss against GPU hours.

4.3 ESTIMATE THE SCORES ONLINE

After task i completes, GPAS updates one moving average, $\hat{s}_i^2 \leftarrow \gamma \hat{s}_i^2 + (1 - \gamma) \|\tilde{G}_i\|_2^2$. Cost-GPAS applies the same update to measured task time. A round-robin warm start observes every task. After computing the proposal, the implementation sets probabilities below $q_{\min}$ to the floor and renormalizes the remaining entries. The floor limits score staleness and large importance weights.

Score error changes sampling efficiency smoothly. Appendix B.4 shows that a relative score error of at most δ increases $V(q)$ by at most a factor of $1/(1 - \delta^2)$. E2 evaluates the proposal with held-out gradients, and E3 compares the online moving average with fresh task scores.

Each completed step records the clipped task gradient before the importance correction. The trainer then multiplies the gradient by λ_I / q_I and passes it to the optimizer. Gradient clipping is applied before the correction so the conditional mean in Equation 4 matches the stated training objective. The experiments use AdamW; changing the optimizer changes the metric, not the correction.

5 EXPERIMENTS

The controlled experiment checks GPAS under the SGD and AdamW metrics. Frozen gradient banks test response normalization, proposal quality, and the AdamW instantiation. Paired MOPD runs measure end-to-end efficiency. A final experiment applies the task sampler to GRPO.

Evidence status. The controlled experiment is complete and reproducible from the released script. The four-teacher MOPD and GRPO runs require a multi-GPU training setup and remain pending in this draft. The protocol below assigns a measurement and a success rule to each planned large-model claim.

5.1 CONTROLLED VALIDATION OF THE SAMPLING RULE

We construct three Gaussian task-gradient distributions with different raw gradient scales, AdamW coordinate scales, and task costs. All methods target $\lambda_i = 1/3$ and apply the correction in Equation 3. We draw 20,000 Monte Carlo trials, with 16 corrected task gradients per trial. The controlled comparison omits the probability floor so its proposals match the closed-form calculation.

Under the identity metric used by SGD, Raw GPAS is optimal and attains calculated and Monte Carlo error ratios of 0.534 relative to uniform sampling. Figure 1 evaluates the same problem in the AdamW metric. The calculated GPAS error ratio is 0.698, and the Monte Carlo ratio is 0.697. The

raw-gradient proposal reaches 7.178 times the observed uniform error because its largest raw coordinates receive a small AdamW scale. Cost-GPAS reaches 0.893 times the uniform time-weighted second-moment proxy. Every corrected proposal has a relative AdamW-scaled mean-gradient error below 2.1%. The controlled experiment checks the estimator and implementation. The LLM study tests score quality, teacher compatibility, and training progress.

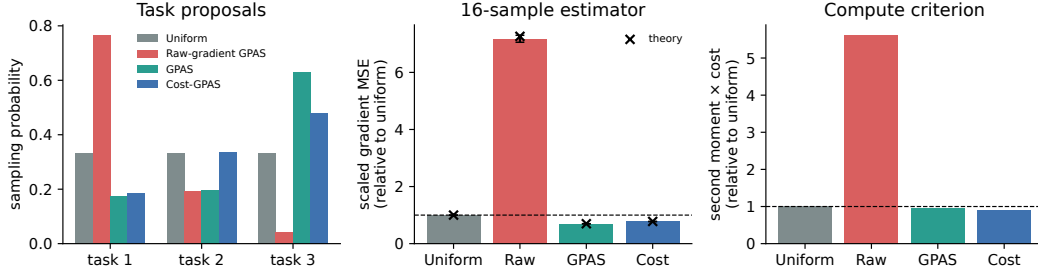


Figure 1: Controlled sampling experiment. Bars show Monte Carlo results normalized by the observed uniform baseline; black crosses show calculated error ratios. Task costs are 5, 2, and 10. Lower values are better in the center and right panels.

5.2 FOUR-TEACHER MOPD SETUP

The planned MOPD study uses a frozen revision of Qwen3-1.7B (Qwen Team, 2025). Four teachers start from the Qwen3 checkpoint and receive separate post-training on mathematics, code, instruction following, and science question answering. Teacher selection occurs before the multi-teacher runs. The release specification requires model revisions, teacher hashes, data splits, and the four capability scores of every teacher.

Each optimizer step samples one task, generates a task-homogeneous batch, calls one frozen teacher, and consumes the rollout once. Every task batch targets 65,536 valid response tokens. Prompt count is estimated from earlier response lengths, and every completed response enters the gradient. The logs record the actual token count and overshoot.

The primary loss uses the sampled-token OPD gradient in Appendix B.1. Each response is averaged by its valid-token count, followed by an average over responses and one shared task-gradient clip. The trainer records the raw and AdamW-scaled scores, applies λ_i/q_i , and calls AdamW. The target mixture is $\lambda_i = 1/4$.

All methods use 600 optimizer steps, an 8,192-token response cap, temperature 1, top- p 1, and disabled thinking mode. A uniform pilot fixes the learning rate, warmup, clipping threshold, batch target, moving-average rate $\gamma = 0.95$, and probability floor $q_i \geq 0.05$. Every paired seed shares a 16-step round-robin warm start, model state, AdamW state, task cursors, and random streams.

5.3 METHODS

Table 1 contains the three main methods and three diagnostic controls. Raw GPAS tests the AdamW scaling. Cost-only tests whether cheap-task preference explains the gains. Uncorrected GPAS measures the objective shift caused by using the proposal as the task weight.

Table 1: Planned MOPD methods. Primary comparisons use five paired seeds; diagnostic controls use three.

Method	Task proposal	Gradient factor	Seeds
Uniform	$q_i = 1/4$	1	41–45
GPAS	$q_i \propto \lambda_i \hat{s}_i$	λ_i/q_i	41–45
Cost-GPAS	$q_i \propto \lambda_i \hat{s}_i / \sqrt{\hat{c}_i}$	λ_i/q_i	41–45
Raw GPAS	$q_i \propto \lambda_i \hat{r}_i$	λ_i/q_i	41–43
Cost-only	$q_i \propto \lambda_i / \sqrt{\hat{c}_i}$	λ_i/q_i	41–43
Uncorrected GPAS	GPAS proposal	1	41–43

Here $\hat{r}_i^2$ tracks $\mathbb{E}\|G_i\|_2^2$. Additional diagnostic runs include a corrected teacher-loss gap sampler, Raw Cost-Aware, round robin, and a full-mixture reference. The full-mixture reference computes four response-normalized microbatches and averages them before one AdamW step. Raw GPAS is the GPAS rule for the SGD identity metric, but here it supplies a cross-metric proposal while every method still trains with AdamW. This isolates the value of AdamW scaling without confounding it with an optimizer change.

5.4 MECHANISM AUDITS

E1: response length and task cost. At steps 0, 150, 300, 450, and 600 of a uniform run, cached rollouts measure each task’s response length, prompt count for a fixed-token batch, token share in a prompt-balanced mixed batch, and GPU time. Cached responses form two gradient estimates: global token averaging and response normalization followed by a weighted task average. E1 tests whether global token averaging follows response length and whether response normalization keeps the explicit coefficient at $1/4$. Timing separates rollout, teacher forward, actor forward/backward, and optimizer work.

E2: corrected-gradient and cost audit. Each checkpoint supplies independent calibration and evaluation gradient banks. The calibration bank builds each proposal. The evaluation bank measures the distance and cosine similarity between the resampled mean gradient and the four-task full-mixture gradient. The evaluation bank also measures the AdamW-scaled estimator error $M(q)$ from Equation 11 and the compute score $J(q)$ defined in Section 4.2. Each result includes its ratio to a proposal built from held-out task gradients and costs. The preregistered predictions are: corrected methods match the full-mixture mean, GPAS improves on Uniform and Raw GPAS in $M(q)$, and Cost-GPAS improves on Uniform, GPAS, and Cost-only in $J(q)$.

E2 also measures teacher–student reverse KL, the fraction of shared tokens in the teacher and student top-16 sets, and the 95th and 99th percentiles of the absolute token log-probability gap. The compatibility metrics show whether a large gradient score comes with a usable teacher signal or an extreme log-ratio tail. Each measure is reported by task and checkpoint.

E3: AdamW score and online tracking. For each task, one model clone applies G_i and another applies a zero gradient from the same model and AdamW state. The difference between the two parameter vectors isolates the task-dependent part of one AdamW step. Task-pair accuracy and Spearman correlation compare raw gradient size and $\|\tilde{G}_i\|_2$ with the measured parameter change. The measured changes also define an exact-step oracle proposal, which bounds the improvement available from a better optimizer metric. A held-out teacher-loss batch records the immediate loss change from each task gradient.

The same cached gradients provide the SGD check: under its identity metric, the raw gradient norm is the exact current-step score up to a shared scalar. This is a metric sanity check rather than a separate end-to-end optimizer comparison.

An excluded uniform pilot compares the online score moving average with fresh gradient-bank scores every 25 steps. A parallel cost audit compares the cost moving average with the next observed task time. The score and cost comparisons test the fixed $\gamma = 0.95$ configuration before the primary runs.

5.5 END-TO-END MOPD EFFICIENCY

Every 25 steps, fresh student rollouts on a fixed prompt set measure the weighted teacher loss

$$L(N) = \sum_i \lambda_i \ell_i(N), \quad (10)$$

where N denotes cumulative valid response tokens or measured GPU hours. The token comparison is GPAS versus Uniform. The GPU-hour comparison is Cost-GPAS versus Uniform and GPAS. Curves use the shortest budget completed by every method in a comparison. An excluded Uniform pilot fixes two teacher-loss targets before the main runs. We report the budget required to reach each target and the area under the loss curve over the shared budget.

Each comparison reports all five paired differences, their mean, a paired effect size, and a 95% paired interval. An efficiency claim requires four of five seeds in the predicted direction, an interval excluding zero, and the matching E2 metric in the predicted direction. We separately verify that the methods use the same number of model evaluations per step and report sampler synchronization overhead.

The run record includes response tokens, teacher-scored tokens, prompts, optimizer steps, GPU hours, wall-clock time, truncation, invalid outputs, sampling probabilities, inverse sampling factors, infrastructure failures, and every task’s teacher loss.

Capability evaluation. The capability evaluation covers the initial student and checkpoints at steps 300 and 600. Metrics are MATH-500 greedy pass@1, a frozen post-cutoff LiveCodeBench pass@1 slice, IFBench strict accuracy, and GPQA-Diamond averaged over four samples. The capability table contains all four scores, their macro average, and the lowest score. The full vector displays the trade-off behind each aggregate.

5.6 TRANSFER TO MULTI-TASK GRPO

The GRPO study uses MATH train, APPS train, and ARC-Challenge train with executable or exact-match rewards (Hendrycks et al., 2021b;a; Clark et al., 2018). The target is $\lambda_i = 1/3$. Each task-homogeneous batch targets 32,768 response tokens and samples eight responses per prompt. Uniform, sequence-level Cost-Aware GRPO, GPAS, and Cost-GPAS share a 12-step round-robin warm start and run for 400 optimizer steps. The Cost-Aware GRPO baseline first samples tasks with $q_i = \lambda_i$, then resamples completed responses inside the selected task in proportion to advantage magnitude divided by the square root of prompt-plus-response length (Mohri et al., 2026). A small uniform mixture gives every completed response positive sampling probability. Importance weights preserve the empirical task objective. GPAS and Cost-GPAS select the task before generation and use the task-level correction in Equation 3.

The primary curves report weighted held-out KL-regularized return over tokens and GPU hours. MATH-500, APPS test pass@1, and ARC-Challenge accuracy provide the task vector. Multi-Task GRPO supplies a capability-oriented reference using its published task-weight rule (Ramesh et al., 2026).

6 DISCUSSION AND CONCLUSION

Multi-task post-training needs separate controls for the target objective and compute allocation. Response-normalized task batches define the unit of the target; inverse sampling preserves its conditional mean. GPAS chooses proposals in an optimizer-induced metric, and Cost-GPAS also uses measured task time. The identity metric recovers the SGD gradient-norm rule; our LLM experiments focus on the lagged AdamW metric.

The controlled experiment supports the local estimator claim. GPAS reduces the measured error by 30.3%, and Cost-GPAS reduces the time-weighted proxy by 10.7%. The constructed comparison shows that coordinate scale can matter. The planned gradient-bank and exact-step audits will measure whether the lagged AdamW score remains useful in LLM training.

Three boundaries guide interpretation. First, the correction preserves the conditional mean of the gradient delivered to the optimizer. Stateful optimizers can still produce different parameter paths under different task orders. Second, the score combines mean scale and batch variation. Compatibility and gradient-bank audits measure whether a large score carries a consistent training signal. Third, Cost-GPAS optimizes an uncentered local compute score. The GPU-hour curves provide the end-to-end efficiency measurement.

The present draft contains controlled estimator evidence and a fixed large-model protocol. Completion of E1–E4 is required for a claim about MOPD training efficiency, and E5 is required for a GRPO transfer claim. The evidence boundary keeps the mathematical result, the MOPD hypothesis, and the eventual capability results distinct.

AI USE STATEMENT

We used generative AI tools for language editing and manuscript organization. The authors reviewed every AI-assisted revision and take responsibility for the final text, claims, proofs, code, and experimental results.

ETHICS STATEMENT

The study evaluates optimization methods for existing language models with established reasoning and instruction-following datasets and automated verifiers. Model and dataset biases can affect the reported capability scores. We record data provenance, invalid outputs, and verifier failures to support careful interpretation of the results.

REPRODUCIBILITY STATEMENT

Section 5 specifies the model family, task setup, update budget, evaluation points, and evaluation protocol. The appendix gives the sampling derivations and implementation details. The repository includes the controlled experiment code and generated figure. The release specification for E1–E5 requires frozen data manifests, model and tokenizer revisions, random seeds, optimizer settings, task-sampling logs, saved parameter and optimizer states, and per-prompt evaluation outputs.

REFERENCES

- Guillaume Alain, Alex Lamb, Chinnadurai Sankar, Aaron Courville, and Yoshua Bengio. Variance reduction in sgd by distributed importance sampling, 2015. URL <https://arxiv.org/abs/1511.06481>.
- Zhao Chen, Vijay Badrinarayanan, Chen-Yu Lee, and Andrew Rabinovich. Gradnorm: Gradient normalization for adaptive loss balancing in deep multitask networks. In *Proceedings of the 35th International Conference on Machine Learning*, pp. 794–803, 2018. URL <https://proceedings.mlr.press/v80/chen18a.html>.
- Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try ARC, the AI2 reasoning challenge, 2018. URL <https://arxiv.org/abs/1803.05457>.
- Nicholas E. Corrado, Wenyuan Huang, and Josiah P. Hanna. Distributionally robust multi-task reinforcement learning via adaptive task sampling, 2026. URL <https://arxiv.org/abs/2605.14350>.
- Huan-ang Gao, Haohan Chi, Yong Yan, Shiyuan Feng, Hanlin Wu, Zheng Jiang, Bingxiang He, Wei-Ying Ma, Ya-Qin Zhang, and Hao Zhou. Open-mopd: Diagnosing and fixing capability imbalance in multi-teacher on-policy distillation, 2026. URL <https://arxiv.org/abs/2608.19098>.
- Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. Measuring coding challenge competence with APPS, 2021a. URL <https://arxiv.org/abs/2105.09938>.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Advances in Neural Information Processing Systems*, 2021b. URL <https://arxiv.org/abs/2103.03874>.
- Angelos Katharopoulos and François Fleuret. Not all samples are created equal: Deep learning with importance sampling. In *Proceedings of the 35th International Conference on Machine Learning*, pp. 2525–2534, 2018. URL <https://proceedings.mlr.press/v80/katharopoulos18a.html>.
- Thibault Lahire. Non-uniform sampling and adaptive optimizers in deep learning. OPT 2023: Optimization for Machine Learning, 2023. URL <https://opt-ml.org/papers/2023/paper20.pdf>.

- Zeman Li, Yuan Deng, Peilin Zhong, Meisam Razaviyayn, and Vahab Mirrokni. Pike: Adaptive data mixing for large-scale multi-task learning under low gradient conflicts. In *Advances in Neural Information Processing Systems*, 2025. URL <https://arxiv.org/abs/2502.06244>.
- Bo Liu, Xingchao Liu, Xiaojie Jin, Peter Stone, and Qiang Liu. Conflict-averse gradient descent for multi-task learning, 2024. URL <https://arxiv.org/abs/2110.14048>.
- Wenhan Ma, Jianyu Wei, Liang Zhao, Hailin Zhang, Bangjun Xiao, Lei Li, Qibin Yang, Bofei Gao, Yudong Wang, Rang Li, Jinhao Dong, Zhifang Sui, and Fuli Luo. Mopd: Multi-teacher on-policy distillation for capability integration in llm post-training, 2026a. URL <https://arxiv.org/abs/2606.30406>.
- Yingzi Ma, Zichen Zhu, Ming Jiang, and Chaowei Xiao. One student, many teachers: Multi-task on-policy distillation via soft-prompt privileged context, 2026b. URL <https://arxiv.org/abs/2607.18293>.
- Clara Mohri, Amir Globerson, Haim Kaplan, Tomer Koren, and Yishay Mansour. Cost-aware learning, 2026. URL <https://arxiv.org/abs/2604.28020>.
- Qwen Team. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.
- Shyam Sundhar Ramesh, Xiaotong Ji, Matthieu Zimmer, Sangwoong Yoon, Zhiyong Wang, Haitham Bou Ammar, Aurelien Lucchi, and Ilija Bogunovic. Multi-task grpo: Reliable llm reasoning across tasks, 2026. URL <https://arxiv.org/abs/2602.05547>.
- Ozan Sener and Vladlen Koltun. Multi-task learning as multi-objective optimization, 2019. URL <https://arxiv.org/abs/1810.04650>.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. URL <https://arxiv.org/abs/2402.03300>.
- Chen Wang, Zhaochun Li, Jionghao Bai, Yining Zhang, Hexuan Deng, Ge Lan, and Yue Wang. Distilled reinforcement learning for llm post-training, 2026a. URL <https://arxiv.org/abs/2607.17247>.
- Rui Wang, Hongru Wang, Yi Chen, Boyang Xue, Tianqing Fang, Wenhao Yu, and Kam-Fai Wong. Demystifying on-policy distillation: Roles, pathologies, and regulations, 2026b. URL <https://arxiv.org/abs/2607.13399>.
- Runzhe Wu, Ankur Samanta, Ayush Jain, Scott Fujimoto, Jeongyeol Kwon, Ben Kretzu, Youliang Yu, Kaveh Hassani, Boris Vidolov, and Yonathan Efroni. Imbalanced gradients in rl post-training of multi-task llms, 2025. URL <https://arxiv.org/abs/2510.19178>.
- Yuan Xie, Jiaqi Song, Xianliang Wang, Ming Lei, Jie Gao, and Jie Wu. Language-specialized multi-teacher on-policy distillation for multilingual llm-based asr, 2026. URL <https://arxiv.org/abs/2608.03610>.
- Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient surgery for multi-task learning, 2020. URL <https://arxiv.org/abs/2001.06782>.

A PRACTICAL TRAINING RECIPE

The recipe uses one complete task batch and one optimizer step per rollout. Task selection occurs before generation, so each step calls one teacher or reward function.

1. Fix the task target. Choose λ from the intended capability priorities and record it before choosing a sampler. For frozen MOPD teachers, λ is the teacher mixture.

2. Define one task gradient. Use a task-homogeneous batch. Average token gradients inside each response and then average responses to form G_i . Apply one shared task-gradient clip. Set a shared valid-token target, and record prompt count, token count, and GPU time.

3. Initialize the sampler. Run a short round-robin warm start. Store a moving average of $\|\tilde{G}_i\|_2^2$ for every task in the selected optimizer metric. This is the raw gradient norm for SGD; the four-task protocol uses AdamW’s stored second moment before the sampling correction. Cost-GPAS also stores a moving average of task time. The protocol uses 16 warm-start steps and $\gamma = 0.95$.

4. Sample and optimize. GPAS sets $q_i \propto \lambda_i \hat{s}_i$. Cost-GPAS sets $q_i \propto \lambda_i \hat{s}_i / \sqrt{\hat{c}_i}$. Apply the probability floor, sample one task, compute and clip G_i , record its score, multiply by λ_I / q_I , and call the optimizer. The four-task protocol uses AdamW and $q_{\min} = 0.05$.

5. Save and audit. Checkpoint the score average, cost average, timestamps, proposal, task cursors, and random state. Log I , q_I , λ_I / q_I , valid tokens, prompt count, GPU time, and the raw and optimizer-metric gradient norms. A large inverse sampling factor or a stale task score triggers inspection of the probability floor.

B DERIVATIONS AND IMPLEMENTATION DETAILS

B.1 SAMPLED-TOKEN OPD

Fix a student-generated prefix z . Write $p_v = \pi_\theta(v \mid z)$ and $\tau_v = T_i(v \mid z)$. The reverse-KL ascent gradient at prefix z is

$$u_i(z) = \sum_v p_v \log \frac{\tau_v}{p_v} \nabla_\theta \log p_v. \quad (11)$$

For a token V sampled from p , the sampled-token gradient is

$$U_i(z, V) = \text{sg} \left[\log \frac{\tau_V}{p_V} \right] \nabla_\theta \log p_V, \quad \mathbb{E}[U_i(z, V) \mid z] = u_i(z). \quad (12)$$

Response normalization averages the prefix gradients according to Equation 1.

B.2 TASK-HOMOGENEOUS GRPO GRADIENT

For task i , GRPO samples response groups from one prompt, computes within-prompt advantages, and applies the standard clipped policy surrogate (Shao et al., 2024). The implementation first averages token terms inside each response and then averages responses inside the task batch. The resulting clipped task gradient is G_i . The GRPO policy ratio handles behavior-policy change inside a task, while λ_i / q_i handles task sampling.

B.3 FROZEN-GRADIENT EVALUATION METRIC

For an evaluation bank, let $\tilde{h} = A_t h$ be the full-mixture mean in the selected optimizer metric, and let $\tilde{g} = A_t \hat{g}$ be the transformed corrected draw. The centered error is

$$M(q) = \mathbb{E} \left\| \tilde{g} - \tilde{h} \right\|_2^2. \quad (13)$$

The experiment estimates h and the expectation from held-out task batches. Calibration batches supply q , so proposal construction and evaluation use separate samples. The LLM evaluation uses the AdamW metric in Equation 5; the controlled experiment also reports the SGD identity metric.

B.4 APPROXIMATE-SCORE PENALTY

Let $q_i^* = \lambda_i s_i / Z$, where $Z = \sum_i \lambda_i s_i$, and let $\hat{q}_i = \lambda_i \hat{s}_i / \hat{Z}$. Write $w_i = \lambda_i s_i / Z$ and $r_i = \hat{s}_i / s_i$. Then

$$\frac{V(\hat{q})}{V(q^*)} = \left(\sum_i w_i r_i \right) \left(\sum_i \frac{w_i}{r_i} \right). \quad (14)$$

If $r_i \in [1 - \delta, 1 + \delta]$, the Kantorovich inequality gives

$$\frac{V(\hat{q})}{V(q^*)} \leq \frac{((1 - \delta) + (1 + \delta))^2}{4(1 - \delta)(1 + \delta)} = \frac{1}{1 - \delta^2}. \quad (15)$$

The result concerns proposal quality for fixed scores. Model drift, observation noise, and the probability floor are measured separately.

B.5 PROBABILITY FLOOR AND SAVED STATE

The online sampler fixes entries below $q_{\min}$ and renormalizes the remaining active entries. A checkpoint stores every task’s score average, cost average, observation count, last-step time, task cursor, and current proposal. Restoring the random state and task cursors reproduces the next task draw and batch.

B.6 ADAMW TASK-STEP MEASUREMENT

Equation 5 uses the previous bias-corrected second moment and omits the current gradient’s effect on AdamW’s denominator. AdamW momentum can also move parameters when the current task gradient is zero. The matched-state measurement in Section 5.4 runs one clone with task gradient G_i and one clone with a zero gradient. Both clones use the same model, AdamW state, learning rate, and moment decay. The difference between the two parameter vectors isolates the increment associated with the current task gradient. For SGD, the corresponding matched-state difference is a shared scalar times G_i , so the identity metric introduces no lagged-preconditioner approximation.